Théorie de la programmation compositionnelle Sémantique, ASCRC, activation réversible, mémoire vivante, synthèse et vérification

Fondement théorique intégré de FulleGenBrain_4

25 octobre 2026

Abstract

Ce document développe et étend une théorie opératoire de la programmation compositionnelle comme fondation de FulleGenBrain_4. Le principe est de construire des comportements et des programmes à partir de composants typés, versionnés, observables et vérifiables, au moyen d'une syntaxe compositionnelle déclarative dont la sémantique, les effets, les coûts, les dépendances et les conditions d'échec sont explicités. Le périmètre va des fonctions pures aux compositions de processus avec état, effets, observation, recherche, récupération, vérification, apprentissage et transformation contrôlée de programmes. La théorie sépare explicitement ASCR (mécanisme existant d'eGenBrain v4, inchangé) et ASCRC (régulation réversible nouvelle). Elle distingue les lois prouvées sous hypothèses formelles, les invariants de conception, les mécanismes observés dans les sources AAM O94.6, et les propriétés qui exigent encore des preuves machine ou des expériences plus larges. Elle traite explicitement la recherche combinatoire, la veille et le réveil des compositions, la mémoire hiérarchique composable, la compression réversible ou approximative qualifiée, et la propagation d'informations sous forme d'événements typés. La mise en veille borne le working set actif mais ne réduit pas, à elle seule, le nombre de candidats possibles : elle est donc combinée à une recherche paresseuse, au filtrage par contrats, au partage, à la déduplication démontrée et à une exploration reprenable. Elle est conçue pour être extensible : une grammaire ou un type de composant insuffisant génère un chantier de formalisation, de construction et de vérification, pas un motif pour abandonner une tâche.

Contents

1 Mandat, terminologie et statut des affirmations 3

2 Examen des artefacts AAM O94.6 et portée des constats 3

3 Objet mathématique central 3

4 Algèbre de programmes compositionnels 4

5 Sémantique et preuves de correction compositionnelle 7

6 Système de types, règles de jugement et sémantique exécutable 8

7 Transactions, effets externes et sémantique de récupération 11

8 Intégrité des traces et portée des empreintes 12

9 Limites logiques générales et obligations d'extension 12

10 Effets, capacités, droits et sûreté de l'exécution 13

11 Résilience : erreurs, reprise, transactions et continuité 13

12 Composition de données, mémoire et provenance 14

13 Mémoire vivante composable : état, compression, veille et réveil 14

14 Synthèse, recherche et transformation de programmes 17

15 Explosion combinatoire : recherche, activation et réutilisation 18

16 ASCRC : régulation réversible de l'activation compositionnelle 19

17 Programmation compositionnelle multimodale et sémantique 22

18 Programmation par composition comme pilier de FulleGenBrain_4 22

19 Modèle du monde : rôle fonctionnel, preuve d'utilité et activation conditionnelle 23

20 Protocole de validation exhaustif et évolutif 25

21 Obligations théoriques restantes et stratégie de fermeture 25

22 Synthèse des garanties et des statuts 26

23 Conclusion 26

1 Mandat, terminologie et statut des affirmations

La programmation compositionnelle est ici une méthode générale permettant de construire une capacité complexe à partir d'unités plus petites dont les contrats et les relations sont explicités. Elle ne se réduit ni à chaîner des fonctions, ni à une syntaxe DSL, ni au fait de ranger des modules dans un catalogue. Une composition complète doit préciser au moins : (i) la structure du programme ; (ii) le typage des flux de données ; (iii) les préconditions et postconditions ; (iv) les effets et ressources ; (v) la sémantique d'exécution ; (vi) la gestion des défaillances ; (vii) la provenance ; (viii) les critères de validation ; (ix) les règles de transformation et de révision.

Les statuts utilisés dans le document sont : SOURCE-OBSERVED (élément retrouvé dans les artefacts examinés), DEFINITION (construction mathématique donnée ici), PROVED-UNDER-ASSUMPTIONS (preuve déductive avec hypothèses identifiées), DESIGN-INVARIANT (règle de conception à faire respecter par l'implémentation), EMPIRICAL-TO-TEST (effet attendu mais non démontré), NOT-CHECKED-BY-PROVER (preuve humaine structurée, pas encore vérifiée par assistant de preuve).

2 Examen des artefacts AAM O94.6 et portée des constats

2.1 Éléments effectivement observés

L'audit livré dans le paquet AAM O94.6 décrit une séparation entre le code du modèle, les preuves, les reçus, les journaux et les oracles de tests. Le démonstrateur compositionnel joint comprend un catalogue de briques et un DSL JSON avec des nœuds `invoke`, `sequence`, `condition` et `map`; les expressions de données prennent notamment la forme de variables, littéraux, listes, objets et opérateurs purs. Le compilateur vérifie les identifiants de composants, la forme des nœuds, la profondeur et le nombre de nœuds, produit un fichier source et un bytecode, puis écrit un manifeste avec empreintes. Le runtime résout les expressions et n'autorise que les identifiants de composants de son catalogue. La composition d'exemple orchestre validation de requête, inventaire de projet, compilation, tests et attachement d'intégrité.

Le paquet de démonstration examiné dans ce travail contient un mini-projet d'éditeur et un ensemble local de cinq tests. Dans l'environnement présent, l'appel initial des tests depuis la mauvaise racine a produit une erreur d'import de module; après lancement depuis le répertoire du projet avec son chemin d'import local, les cinq tests ont réussi. Ce fait valide uniquement les cinq tests de ce mini-projet et non la totalité d'AAM, ni la théorie ici élaborée. Le reçu AAM de la matrice P8 rapporte séparément 92 fichiers de test avec 449 réussites cumulées, zéro échec et zéro timeout dans son environnement d'audit; ce résultat est attribué à ce reçu et à ses limites, il n'est pas présenté comme une exécution effectuée dans le présent travail.

2.2 Ce que ces constats autorisent à conclure

Ils fournissent un point d'appui pratique pour la composition déclarative, le registre de composants, la compilation, les manifests, le traçage et les portes de qualité. Ils ne suffisent pas à démontrer l'exhaustivité de la théorie, la correction générale du compilateur, la sûreté de tout composant, la complétude de la synthèse ou la généralisation à un domaine arbitraire. Le présent document formalise donc les obligations mathématiques et d'ingénierie qui sont nécessaires pour élargir ce noyau.

3 Objet mathématique central

3.1 Composant typé

Définition 3.1 (Contrat d'un composant)**.** Un composant est le tuple

$$c = (id, ver, A, B, P, Q, E, R, \delta, \pi, V),$$

où id est l'identifiant stable, ver la version, A et B les types d'entrée et de sortie, P la précondition, Q la postcondition, E l'ensemble des effets possibles, R le profil de ressources, δ la sémantique d'exécution, π le schéma de provenance et V le vérificateur ou protocole d'évaluation associé. Les types peuvent être algébriques, dépendants d'un contexte, structurés ou enrichis de contraintes d'unités et d'effets.

Un contrat doit distinguer l'interface syntaxique de la garantie sémantique. Déclarer qu'un composant transforme A en B ne prouve pas qu'il le fait toujours. Un niveau de garantie est enregistré dans les métadonnées : spécifié, compilé, testé localement, vérifié formellement sous hypothèses, ou observé en production. Ces niveaux ne se substituent pas les uns aux autres.

3.2 Types de résultat

Une exécution renvoie non seulement une valeur, mais un résultat typé :

$$Y = \mathsf{Success}(v, w) \mid \mathsf{Partial}(v, w) \mid \mathsf{Unresolved}(U, w) \mid \mathsf{Rejected}(r, w) \mid \mathsf{Failed}(e, w) \mid \mathsf{Cancelled}(w),$$

où w est une trace de provenance, U un ensemble structuré d'alternatives, r une raison de refus et e une description d'erreur. Une chaîne ne doit pas convertir silencieusement une sortie partielle ou non résolue en succès. Les résultats peuvent être enrichis sans casser les anciens consommateurs si leurs schémas sont versionnés.

3.3 État, événements et trace

L'état d'exécution est $s = (D, K, M, B, C)$: données locales D, contexte et connaissances accessibles K, mémoire durable M, budgets restants B et capacités autorisées C. Une étape applique une transition

$$\langle p, s\rangle \longrightarrow \langle p', s', r, e\rangle,$$

où p est le programme restant, r le résultat partiel et e l'événement de trace. La trace est une séquence append-only d'événements identifiés, horodatés, versionnés et chaînés par hash si l'exigence d'intégrité le requiert. Un hash signale un changement de contenu; à lui seul, il ne prouve ni la vérité de l'événement ni l'identité de son émetteur.

4 Algèbre de programmes compositionnels

4.1 Syntaxe abstraite extensible

Définition 4.1 (Langage de composition)**.** Une grammaire de base est donnée par :

$$\begin{aligned} p ::= {} & \mathsf{Id} \mid \mathsf{Call}(c, e) \mid \mathsf{Seq}(p_1, \ldots, p_n) \mid \mathsf{Par}(p_1, \ldots, p_n) \\ & \mid \mathsf{If}(g, p_t, p_f) \mid \mathsf{Map}(p, e, n) \mid \mathsf{Fold}(p, e, a) \\ & \mid \mathsf{Loop}(g, p, I) \mid \mathsf{Choose}(p_1, \ldots, p_n) \mid \mathsf{Verify}(p, v) \\ & \mid \mathsf{Observe}(q) \mid \mathsf{Probe}(Q) \mid \mathsf{Recover}(p, \rho) \\ & \mid \mathsf{Txn}(p, \kappa) \mid \mathsf{Checkpoint}(p, k) \mid \mathsf{Revoke}(d) \\ & \mid \mathsf{Compose}(m, \theta) \mid \mathsf{Transform}(p, \tau) \mid \mathsf{Fix}(x, p) \\ & \mid \mathsf{Select}(\pi, p_1, \ldots, p_n) \mid \mathsf{Synthesize}(G, D) \\ & \mid \mathsf{Hibernate}(c) \mid \mathsf{Wake}(c, q) \mid \mathsf{Compress}(m, k) \\ & \mid \mathsf{Summarize}(m, k, \epsilon) \mid \mathsf{Restore}(k) \\ & \mid \mathsf{EmitWave}(w) \mid \mathsf{ReceiveWave}(w) \mid \mathsf{ResumeSearch}(k). \end{aligned}$$

Les formes non présentes dans un DSL existant sont des extensions théoriques à spécifier et implémenter, pas des capacités attribuées rétroactivement au démonstrateur. La grammaire doit être extensible par modules typés et versionnés, avec analyse syntaxique, validation sémantique, compilation et tests différentiels.

4.2 Sémantique contractuelle des opérateurs de recherche et de mémoire

Les constructeurs transversaux sont des opérations du langage, pas des effets implicites du runtime. La table ci-dessous fixe leur résultat contractuel minimal et les contrôles de sûreté. Une implémentation peut raffiner ces contrats, mais ne peut ni supprimer la provenance, ni transformer un résultat incertain en succès certifié.

Opérateur	Sémantique / résultat	Conditions et invariants minimaux
Choose$(p_1, \ldots, p_n)$	Expose l'ensemble des alternatives admissibles; ne choisit pas silencieusement une gagnante.	Alternatives bien typées; stratégie de résolution explicite si une seule voie doit être exécutée.
Select	Applique la politique versionnée π aux seules alternatives admissibles et journalise le choix.	Score heuristique distinct d'une preuve; contraintes dures et portes de sûreté non contournables.
Synthesize(G, D)	Produit un front ou un flux reprenable de PROGRAM-CANDIDATE qui répond à l'objectif G sur les données D.	Grammaire, budgets et données versionnés; contre-exemples conservés; aucun candidat autocertifié.
Hibernate(c)	Décharge la représentation active en gardant le payload exact récupérable, le hash et les dépendances.	Refus si épinglé, loué, en transaction ou requis par une révocation; checkpoint persistant avant déchargement.
Wake(c, q)	Restaure c à la demande de contexte q, puis produit ACTIVE, STALE ou un refus explicite.	Vérification d'intégrité, de version, de contexte, de droits et revalidation des dépendances; le chargement seul ne valide rien.
Compress(m, k)	Encode m dans un format k avec reconstruction canonique sans perte.	Round-trip et hash contrôlés; ratio mesuré; provenance/version conservées.
Summarize(m, k, ϵ)	Produit une vue dérivée avec perte contrôlée dans un domaine de questions défini.	Source exacte conservée; ϵ et métrique qualifiés par domaine; la vue ne remplace jamais une preuve source.
Restore(k)	Réhydrate un checkpoint ou objet stocké en vérifiant sa cohérence.	Schéma et versions compatibles; intégrité, dépendances et effets externes réconciliés ou déclarés ambigus.
EmitWave(w)	Persiste un événement typé dirigé vers des abonnés admissibles.	Identifiant idempotent, provenance source/version, priorité, TTL/fan-out et backpressure.
ReceiveWave(w)	Valide, déduplique et route une vague vers un handler déclaré.	Schéma/version/source contrôlés; handler idempotent ou sémantique de replay explicite; révocations prioritaires.
ResumeSearch(k)	Restaure un front de recherche après budget ou interruption.	AST/grammaire, signature, versions, graines, statistiques et budget cohérents; migration ou rejet si schéma incompatible.

Opérateur	Sémantique / résultat	Conditions et invariants minimaux
$\mathsf{Checkpoint}(p, k)$	Capture un état cohérent du programme, du front et des effets observables.	Point de cohérence explicite; les effets externes non idempotents exigent réconciliation/compensation, pas un simple rollback mémoire.
$\mathsf{Revoke}(d)$	Révoque une preuve, version ou dépendance et marque ses descendants à réévaluer.	Propagation transitive traçable; aucun descendant dépendant ne reste certifié sans nouvelle justification.
$\mathsf{Txn}(p, \kappa)$	Exécute une transaction selon la politique κ et rend le résultat final ou partiel explicite.	Atomicité limitée au domaine garanti; les effets distribués nécessitent saga, idempotence ou réconciliation.

Portée. Ces contrats spécifient une sémantique cible et ses conditions d'admissibilité. Ils ne déclarent pas que tous les constructeurs sont déjà implantés dans un runtime unique. L'état de couverture entre théorie, architecture, code et tests est conservé dans la matrice FulleGenBrain_4 et dans le registre de preuves.

4.3 Composition séquentielle

Pour $f : A \to B$ et $g : B \to C$, la composition $g \circ f : A \to C$ est bien typée si la sortie de f correspond à l'entrée de g et si les préconditions sont satisfaites. Au niveau des contrats, une règle suffisante est :

$$\{P\}\ f\ \{Q\}, \quad Q \Rightarrow P_g, \quad \{P_g\}\ g\ \{R\} \quad \Longrightarrow \quad \{P\}\ g \circ f\ \{R\}.$$

La condition $Q \Rightarrow P_g$ porte sur les états, les types, les unités, les effets et les hypothèses de contexte, pas seulement sur la forme de la donnée.

4.4 Composition parallèle et ordre

La composition parallèle ne peut être déclarée indépendante qu'après analyse des effets. Soient R_i les zones lues, W_i les zones écrites et X_i les effets externes du composant i. Une condition conservatrice d'indépendance est

$$W_i \cap (R_j \cup W_j) = \varnothing, \quad W_j \cap (R_i \cup W_i) = \varnothing,$$

avec absence de conflit d'effets externes et absence d'interférence par ressources partagées. Sous ces conditions, et si les sémantiques locales sont déterministes sur l'état observé, les deux ordres peuvent être équivalents relativement à la projection d'observation choisie. Sans ces conditions, l'ordonnancement fait partie de la sémantique; permuter deux nœuds est une transformation non autorisée par défaut.

4.5 Identité, associativité et lois de réécriture

L'identité Id ne change ni l'état observable ni le résultat, hormis les événements de trace explicitement déclarés. La séquence est associative au niveau de la composition des fonctions partielles pures; pour les processus avec effets, l'équivalence dépend de la règle d'observation des traces, des frontières de transaction et de l'ordre des exceptions. Toute loi d'optimisation doit donc porter ses hypothèses et son domaine d'équivalence. L'optimiseur ne peut pas supprimer une étape d'audit, une vérification ou un effet simplement parce que sa valeur de sortie paraît inutilisée.

4.6 Branchement, collection, réduction et récursion

Un branchement exige un prédicat typé et une branche de repli explicitée. Un Map doit déclarer le type de chaque élément, la politique d'erreur par élément et la limite de ressources; un Fold doit déclarer le type de l'accumulateur et l'invariant de réduction. Une boucle doit avoir un invariant I, une condition d'arrêt et, pour prouver la terminaison, une fonction de rang μ à valeurs dans un ordre bien fondé, telle que chaque itération non terminale diminue strictement μ. Les programmes récursifs sans preuve de terminaison peuvent être exécutés avec budget, mais leur terminaison ne doit pas être prétendue.

5 Sémantique et preuves de correction compositionnelle

5.1 Sémantique dénotationnelle et opérationnelle

Une composante pure peut être interprétée par une fonction partielle $[\![c]\!] : A \rightharpoonup B$. Un programme avec effets est plutôt une transition sur état et trace :

$$[\![p]\!] : (s, x) \mapsto (s', y, \mathcal{T}, w),$$

avec résultat non garanti si la précondition est violée, une capacité manque ou une ressource s'épuise. Les fonctions partielles et transitions ne doivent pas être ramenées à la même abstraction lorsqu'il existe des écritures, appels distants, horloges, aléatoire, observation du monde ou intervention humaine.

Proposition 5.1 (Correction de la composition séquentielle)**.** *Supposons que les triples de Hoare $\{P\}f\{Q\}$ et $\{Q\}g\{R\}$ soient valides dans une sémantique commune, que l'exécution de f termine sur toute entrée satisfaisant P, et qu'aucune interférence externe ne modifie l'état entre la postcondition Q et l'évaluation de la précondition de g. Alors $\{P\}g \circ f\{R\}$ est valide.*

Proof. Soit un état initial s satisfaisant P. Par la validité du premier triple et la terminaison supposée, l'exécution de f donne un état s_1 satisfaisant Q. Le second triple s'applique donc à s_1 et donne un état final satisfaisant R. Ainsi toute exécution du composé conforme aux hypothèses satisfait R. □

Cette preuve est mathématique et conditionnelle; elle n'est pas une preuve machine du compilateur AAM ni d'une implémentation particulière.

Proposition 5.2 (Composition par contrat)**.** *Soient $c_1, \ldots, c_n$ des composants dont les contrats sont prouvés dans un même système de types. Si chaque connexion transmet une sortie satisfaisant les préconditions du composant suivant, si toutes les capacités exigées sont disponibles et si les effets sont sérialisés conformément à leur contrat, alors la composition séquentielle satisfait la composition des postconditions et conserve la trace de chaque étape.*

Proof. Par induction sur n. Le cas $n = 1$ est le contrat du composant. Pour $n + 1$, l'hypothèse d'induction fournit la postcondition de la composition des n premiers. La condition de raccord rend cette postcondition suffisante pour la précondition du composant $n+1$; la proposition de correction séquentielle achève l'étape inductive. □

Proposition 5.3 (Équivalence sûre d'une composition parallèle)**.** *Supposons deux composants déterministes f et g dont les effets sont indépendants au sens de la section précédente, qui observent et modifient des parties disjointes de l'état, et dont les traces sont comparées après projection sur les événements pertinents au contrat. Alors f puis g et g puis f produisent le même état final projeté et la même valeur projetée.*

Proof. Comme f ne modifie aucune donnée lue ou écrite par g, l'application de g ne dépend pas du fait que f ait déjà exécuté; symétriquement pour f. Les mises à jour sur des composantes disjointes de l'état commutent. Les valeurs produites sont donc identiques et les états finaux diffèrent au plus sur l'ordre d'événements que la projection de trace a explicitement choisi d'ignorer. □

5.2 Vérification indépendante et certificats

Un certificat de composition contient au minimum : hash de la spécification source normalisée, versions de composants et du vérificateur, contrats utilisés, graphe de dépendances, configuration d'exécution, résultat, trace, hypothèses et verdict. Le compilateur et le vérificateur ne doivent pas partager silencieusement la même logique critique lorsque l'indépendance est une exigence. Une compilation réussie signifie que le programme a satisfait le contrôle de compilation annoncé; ce n'est pas une preuve de correction métier. Un hash lie des octets à une empreinte; il ne valide pas le contenu.

6 Système de types, règles de jugement et sémantique exécutable

6.1 Types et environnements

Les types du noyau sont générés par la grammaire

$$\begin{aligned}\tau ::= {} & \mathbf{Unit} \mid \mathbf{Bool} \mid \mathbf{Int} \mid \mathbf{Real} \mid \mathbf{String} \mid \mathbf{Any} \\ & \mid \mathbf{List}(\tau) \mid \mathbf{Map}(\tau,\tau) \mid \mathbf{Pair}(\tau,\tau) \\ & \mid \mathbf{Sum}(\tau,\tau) \mid \mathbf{Ref}(\tau) \mid \mathbf{Record}(S).\end{aligned}$$

Les enregistrements S ont un schéma versionné. Les types numériques peuvent être décorés d'une dimension d'unité; les références peuvent être décorées par leur portée et leur version. Le type **Any** n'est permis qu'à la frontière d'entrée/sortie opaque et déclenche un contrôle dynamique avant toute opération qui demande un type concret.

Γ est l'environnement des variables, Σ le registre versionné des signatures de composants, et K l'ensemble des capacités d'exécution. Un jugement d'expression est $\Gamma \vdash e : \tau$. Un jugement de programme est

$$\Gamma; \Sigma; K \vdash p : A \Rightarrow B \,!\, E \; [P, Q] \text{ cost} \leq r,$$

ce qui signifie : sous les variables disponibles, les signatures versionnées et les capacités K, le programme transforme une entrée A en résultat B, peut produire les effets E, est soumis à la précondition P, vise la postcondition Q et déclare une borne de ressources r. Le jugement n'affirme ces propriétés que si les contrats des composants référencés sont eux-mêmes valides à leur niveau de preuve déclaré.

6.2 Règles de typage des expressions

Les littéraux sont typés par leur type concret. Si $x : \tau \in \Gamma$, alors $\Gamma \vdash x : \tau$. Pour les listes, si chaque élément e_i a le type τ, la liste a le type $\mathbf{List}(\tau)$. Pour un opérateur pur *op* avec signature $\tau_1 \times \cdots \times \tau_n \to \tau$, les prémisses $\Gamma \vdash e_i : \tau_i$ donnent $\Gamma \vdash op(e_1, \ldots, e_n) : \tau$. La règle est refusée si un opérateur n'est pas dans le registre ou si les types ne satisfont pas la signature. Une coercition éventuelle est un opérateur explicite avec sa propre sémantique, jamais une conversion silencieuse.

6.3 Règles de typage des programmes

Identité.

$$\frac{}{\Gamma; \Sigma; K \vdash \mathsf{Id} : A \Rightarrow A \,!\, \varnothing \; [\mathsf{true}, \mathsf{true}]}.$$

Appel. Si $\Sigma(c) = (A, B, P_c, Q_c, E_c, r_c)$, $\Gamma \vdash e : A$, K autorise E_c, et $P \Rightarrow P_c(e)$, alors

$$\frac{\Sigma(c) = (A, B, P_c, Q_c, E_c, r_c) \quad \Gamma \vdash e : A \quad E_c \subseteq K \quad P \Rightarrow P_c(e)}{\Gamma; \Sigma; K \vdash \mathsf{Call}(c, e) : A \Rightarrow B \,!\, E_c \; [P, Q_c]}.$$

La postcondition de l'appel est instanciée par les entrées et sorties du composant. Une capacité manquante rend l'appel non admissible pour cette exécution; elle ne transforme pas l'effet en pur.

Séquence. Si $p_1 : A \Rightarrow B\ !E_1[P, Q_1]$ et $p_2 : B \Rightarrow C\ !E_2[P_2, Q_2]$ sont typés et $Q_1 \Rightarrow P_2$, alors

$$\frac{\Gamma; \Sigma; K \vdash p_1 : A \Rightarrow B\ !E_1[P, Q_1] \quad \Gamma; \Sigma; K \vdash p_2 : B \Rightarrow C\ !E_2[P_2, Q_2] \quad Q_1 \Rightarrow P_2}{\Gamma; \Sigma; K \vdash \mathsf{Seq}(p_1, p_2) : A \Rightarrow C\ !(E_1 \cup E_2)[P, Q_2]}.$$

Branchement. Si g est booléen et les deux branches ont les mêmes types d'entrée/sortie et une postcondition commune Q, le branchement a l'union de leurs effets. La postcondition s'applique à la branche sélectionnée, pas à la branche non exécutée.

Map. Si $p : A \Rightarrow B$ et $e : \mathbf{List}(A)$, alors $\mathsf{Map}(p, e, n)$ produit $\mathbf{List}(B)$ uniquement si la politique sur les erreurs, le maximum n et la consommation de ressources sont déclarés. En politique StopOnError, le résultat global n'est pas Success si une itération échoue; en politique Collect, le type de sortie doit être un type somme qui préserve séparément résultats et erreurs.

Vérification. $\mathsf{Verify}(p, v)$ ne transforme pas arbitrairement un résultat : elle produit un type de résultat distinguant validation, rejet et vérification indéterminée. La promotion épistémique dépend ensuite du contrat du vérificateur.

Récupération. $\mathsf{Recover}(p, q)$ est typé si la sortie de q respecte le contrat minimal exigé par l'appelant lorsque p échoue. Une récupération ne peut être considérée comme sûre que si les effets de p sont purs, transactionnels, idempotents ou compensables, et que sa politique de restauration est spécifiée.

Boucle. $\mathsf{Loop}(g, p, I)$ exige un garde booléen et un invariant I. Pour prétendre à la terminaison, le contrat ajoute une fonction de rang μ qui décroît strictement dans un ordre bien fondé à chaque itération non terminale.

Parallélisme. $\mathsf{Par}(p_1, p_2)$ n'est admissible que si les effets se composent, que l'interférence est exclue ou résolue par une politique formelle, et que le mécanisme de fusion des sorties est défini. L'absence de preuve d'indépendance conduit à conserver un ordre séquentiel ou à demander un raffinement des contrats.

6.4 Sémantique à grands pas

La relation d'exécution est

$$\langle p, s \rangle \Downarrow \langle y, s', \mathcal{T}, c \rangle,$$

où y est un résultat typé (succès, partiel, non résolu, rejet, échec ou annulation), s' l'état final, $\mathcal{T}$ la suite des événements et c le coût cumulé. La trace inclut les appels de composants, branches, probes, validations, effets, erreurs, récupérations et décisions de budget.

Appel. L'évaluation d'un appel suit les phases strictes : résolution du composant et de sa version; résolution des expressions; validation du type d'entrée et des préconditions; vérification des capacités; exécution; validation du type de sortie et de la postcondition; écriture de l'événement de résultat. Si une phase échoue, la sémantique renvoie un statut explicite et conserve les événements déjà survenus. Les effets externes déjà réalisés ne sont pas réputés annulés.

Séquence. Si p_1 produit $\mathsf{Success}(v)$ et un état s_1 satisfaisant le contrat de raccord, alors p_2 s'exécute sur s_1. Si p_1 renvoie un statut d'échec ou de rejet, la politique par défaut propage ce statut et n'exécute pas p_2. Une politique de continuation est possible, mais doit être indiquée dans l'AST et spécifier comment les résultats partiels sont agrégés.

Branchement. Le garde est évalué à partir de l'état courant, son résultat est consigné dans la trace, puis une seule branche est exécutée. Une évaluation de garde indéterminée ne choisit pas une branche par défaut : elle produit un statut non résolu ou appelle une procédure de clarification explicitement déclarée.

Map et Fold. L'ordre, les limites, la politique d'erreur et l'agrégation sont partie intégrante de la sémantique. L'environnement local d'un élément est restauré après son traitement, sauf si le contrat déclare expressément une réduction d'état. Un fold doit préciser son accumulateur initial et sa loi de réduction; l'associativité de la réduction n'est pas présumée.

Vérification. Une vérification produit l'un des statuts VALID, INVALID ou UNKNOWN, chacun accompagné d'un certificat, d'un contre-exemple ou d'une raison d'insuffisance. Seul le statut VALID, dans la portée déclarée et après le contrôle d'intégrité du certificat, peut servir de preuve positive pour le contrat concerné.

Récupération. Le runtime sauvegarde le sous-état contrôlé avant d'exécuter le corps. Si le corps échoue, les mutations réversibles sont annulées ou compensées, l'événement de récupération est tracé, puis le repli est exécuté. Si un effet irréversible est en cours de réconciliation, le statut global doit le signaler; la restauration mémoire seule ne suffit pas.

6.5 Progrès, préservation et sûreté de typage

Théorème 6.1 (Sûreté du sous-langage pur)**.** *Considérons le fragment fini composé de* Id*,* Call*,* Seq*,* If*,* Map *sur une liste finie et* Verify*, à l'exclusion de* Loop*,* Fix*, du parallélisme et de tout effet externe. Supposons un registre fini de composants purs, totaux et conformes à leurs signatures, des opérateurs purs totaux sur leurs domaines typés, des gardes et vérificateurs totaux, ainsi qu'une grammaire correctement vérifiée. Si $\Gamma \vdash p : A \Rightarrow B$ et si l'entrée x a le type A, alors l'exécution termine avec un résultat de type B ou avec une erreur typée explicitement représentée par la sémantique; elle ne se bloque pas sur une opération de type inconnu.*

Proof. Par induction structurale sur p. L'identité renvoie son entrée. L'appel vérifie son entrée, puis le composant total fournit une sortie du type annoncé. Pour la séquence, le type de sortie du premier programme fournit le type requis au second. Le branchement choisit une branche dont le résultat est du type commun. Le map traite un nombre fini d'éléments et applique l'hypothèse d'induction à chacun. Le vérificateur est total et renvoie l'un des cas du type-somme déclaré. Les opérations de type invalide sont arrêtées par la vérification et deviennent des erreurs typées; aucune règle du fragment ne crée une opération non représentée par la sémantique. □

Le théorème ne couvre pas les effets externes, les programmes infinis, ni les composants dont l'implémentation viole leur contrat. Ces cas exigent les règles d'effets, de terminaison et de résilience qui suivent.

Théorème 6.2 (Terminaison conditionnelle d'une boucle)**.** *Supposons que l'invariant I soit vrai avant la boucle, qu'il soit préservé par chaque exécution non terminale du corps, que le garde et le corps terminent à chaque itération, et qu'une mesure μ décroisse strictement à chaque itération tout en appartenant à un ordre bien fondé. Alors la boucle termine et laisse un état satisfaisant l'invariant et la négation du garde.*

Proof. Un ordre bien fondé ne contient aucune suite infinie strictement décroissante; la mesure ne peut donc décroître indéfiniment. À l'itération finale, le garde est faux. L'invariant est vrai par induction sur le nombre fini d'itérations et demeure vrai à la sortie. □

6.6 Lois de transformation et équivalence comportementale

Soient p et q deux programmes. Leur équivalence dépend d'une observation contractuelle Obs :

$$p \simeq_{Obs} q \iff \forall x \in D,\ Obs([\![p]\!](x)) = Obs([\![q]\!](x)).$$

Le domaine D, les versions de composants, l'environnement et les effets observés sont partie intégrante de cette affirmation. Une équivalence limitée à des entrées finies testées est notée $\mathsf{EquivalentOn}(D_{test})$, pas $\simeq_{Obs}$ universelle. Les transformations automatiques exigent une preuve d'équivalence dans le sous-langage pris en charge ou un rapport de tests bornés accompagné de sa portée.

Proposition 6.3 (Préservation d'une transformation certifiée)**.** *Si une transformation T fournit un certificat valide démontrant $p \simeq_{Obs} T(p)$ sous des hypothèses H, et si le contexte d'exécution satisfait encore H, remplacer p par $T(p)$ préserve les propriétés observables définies par Obs dans ce contexte.*

Proof. Par définition de l'équivalence observée, les résultats observables de p et $T(p)$ sont égaux pour toutes les entrées du domaine D sous H. Le remplacement conserve donc les propriétés exprimées uniquement par Obs sur ce domaine. Aucun comportement extérieur à D ni propriété absente de Obs n'est déduit. □

6.7 Correspondance interpréteur/compilateur

Un compilateur est correct si l'exécution du programme compilé et celle de son AST source sont reliées par une relation de simulation préservant les résultats, les effets autorisés et la trace contractuelle. Le but formel est :

$$\begin{aligned}\forall p, s, \quad \mathsf{Valid}(p) \Rightarrow\ & \mathrm{Obs}([\![p]\!]_{\mathrm{interp}}(s)) \\ =\ & \mathrm{Obs}([\![\mathrm{Compile}(p)]\!]_{\mathrm{run}}(s)).\end{aligned}$$

Cette formule est une obligation de preuve, pas un résultat déclaré acquis dans le présent dossier. Elle se ferme par (i) sémantique formelle des nœuds; (ii) preuve par induction sur l'AST pour l'émission du code; (iii) tests différentiels générés; (iv) tests de rejet négatifs; (v) contrôle d'empreintes et de versions. Les tests différentiels ne remplacent pas la preuve générale mais peuvent détecter un contre-exemple à une version du compilateur.

7 Transactions, effets externes et sémantique de récupération

7.1 État transactionnel

Une transaction est $\mathsf{Txn}(p, \kappa)$ où κ définit les frontières de commit, rollback et compensation. Son état comprend un journal de mutations préparées, les effets confirmés et les clés d'idempotence. Le commit est autorisé lorsque les préconditions/postconditions du programme et les contrôles requis réussissent. En cas d'échec avant commit, les mutations locales transactionnelles sont annulées. Les effets externes non transactionnels doivent être inscrits au journal comme irréversibles ou soumis à une action compensatoire.

7.2 Reprise et sémantique de livraison

Après interruption, un checkpoint est accepté seulement si ses hashes, versions, schémas et séquence d'événements sont cohérents. Une opération externe avec identifiant stable et accusé de réception connu peut être marquée confirmée; une opération non idempotente dont l'issue est inconnue doit être réconciliée avant nouvel envoi. Les garanties « au moins une fois » et « au plus une fois » ne sont pas équivalentes; « exactement une fois » requiert une déduplication durable ou un protocole coopérant avec le système externe. Il est interdit de revendiquer exactement une fois sur la seule base d'un checkpoint local.

Proposition 7.1 (Reprise idempotente)**.** *Supposons qu'une étape f ait un identifiant d'opération persistant, que le destinataire conserve durablement les clés déjà exécutées, et qu'un replay avec la même clé renvoie le résultat mémorisé sans répéter l'effet. Alors la reprise après crash peut réexécuter f avec la même clé sans dupliquer l'effet externe, sous réserve que la conservation de la clé et de son résultat soit atomique côté destinataire.*

Proof. Lors du premier traitement, le destinataire enregistre le couple clé/résultat avec l'effet. Lors du replay, la même clé est reconnue avant l'exécution de l'effet; le résultat mémorisé est renvoyé. La condition d'atomicité empêche un état dans lequel l'effet est réalisé mais la clé est perdue. Ainsi le replay n'ajoute pas de seconde occurrence de l'effet dans les hypothèses données. □

7.3 Cohérence des checkpoints

Le checkpoint doit contenir ou référencer : id et version de la composition; état sérialisé; état des variables; événements de trace et dernier hash confirmé; opérations externes avec leur état de réconciliation; versions des composants; budget; artefacts et dépendances. Le redémarrage rejoue uniquement les étapes dont les effets le permettent et dont les préconditions sont encore vraies. Les événements non confirmés sont recontrôlés; une donnée périmée déclenche l'invalidation des descendants.

8 Intégrité des traces et portée des empreintes

La trace est chaînée par $h_0 = H(\text{seed})$ et $h_i = H(\text{Canon}(e_i, h_{i-1}))$, où H est un hash cryptographique et Canon une sérialisation canonique versionnée. Toute modification d'un événement ou de son prédécesseur entraîne une incohérence des hashes suivants, à moins d'une collision ou d'une réécriture complète du suffixe. La chaîne n'est pas une signature : pour attribuer la trace à un acteur, il faut une clé, une signature ou une attestation indépendante. Le manifeste relie les artefacts, versions et paramètres mais ne certifie pas à lui seul la correction sémantique.

9 Limites logiques générales et obligations d'extension

Un ensemble fini d'exemples peut être expliqué par plusieurs programmes distincts. L'identifiabilité dépend du domaine, des contraintes, des interventions et de la classe de candidats; la composition ne supprime pas cette propriété logique, elle rend explicites les briques et les hypothèses utilisées. Quand les données ne distinguent pas les candidats, le système doit conserver le sous-ensemble compatible et rechercher une observation qui les sépare. Si la grammaire de candidats ne représente pas la fonction recherchée, elle doit être enrichie par une nouvelle forme de programme, avec preuve de typage, analyse d'effets, tests et trace.

Pour qu'une extension s'intègre, elle doit clôturer un dossier contenant : définition mathématique; syntaxe/AST; règles de typage; sémantique opérationnelle; effets/capacités; préconditions/postconditions; argument de correction; conditions de terminaison; coûts et limites d'exécution; stratégie de panne/reprise; tests positifs et négatifs; version et compatibilité; certificats/provenance; comparaison avec les voies existantes. Une entrée de backlog seule ne satisfait pas cette obligation; elle doit être transformée en sous-projet jusqu'au niveau de formalisation et d'implémentation requis pour son contrat d'intégration.

10 Effets, capacités, droits et sûreté de l'exécution

10.1 Système d'effets

Chaque composant annonce ses effets dans un ensemble typé. Un exemple de vocabulaire d'effets est

$$E = \{\mathsf{Read}(p), \mathsf{Write}(p), \mathsf{Network}(h), \mathsf{Clock},$$
$$\mathsf{Random}, \mathsf{Observe}(t), \mathsf{Execute}(k)\}.$$

Les effets sensibles exigent une capacité détenue, un périmètre, une politique et une trace. L'absence de capacité est une condition de refus ou de recherche d'une voie alternative autorisée, et non une permission implicite.

10.2 Frontière entre code et données

La source compositionnelle est une représentation de programme, pas du code arbitraire déguisé en donnée. Le compilateur analyse le schéma, résout les identifiants dans un registre, contrôle les expressions, puis compile le programme validé. Les littéraux, noms de composants, attributs et chemins ne sont pas équivalents à du code exécutable libre. Toute extension de grammaire qui introduit un nouveau moyen d'exécution doit déclarer son modèle de menace, ses capacités requises et ses tests adversariaux.

10.3 Confinement sans fermeture évolutive

La sécurité d'exécution n'exige pas que le langage demeure figé. Elle exige que chaque nouvelle construction passe par le cycle : spécification, typage, analyse des effets, tests négatifs, revue du compilateur, vérification du manifeste, contrôle de compatibilité, déploiement versionné et possibilité de retour arrière. Si une tâche dépasse la grammaire actuelle, le travail correct consiste à spécifier puis ajouter l'abstraction manquante avec ses obligations de sûreté, plutôt qu'à laisser le besoin hors du modèle.

11 Résilience : erreurs, reprise, transactions et continuité

11.1 Taxonomie des défaillances

Les erreurs sont classées au minimum en : entrée invalide, précondition non satisfaite, contrat de sortie rompu, composant absent, permission insuffisante, indisponibilité temporaire, timeout, manque de budget, état conflictuel, corruption d'artefact, échec de vérification, résultat indéterminé, et erreur de l'orchestrateur. Les catégories déterminent une action de reprise; une exception générique ne suffit pas à sélectionner une stratégie.

11.2 Politiques de reprise

Un composant peut déclarer une politique parmi : aucun nouvel essai, nouvel essai borné, nouvel essai avec backoff, stratégie alternative, récupération depuis checkpoint, compensation de l'effet, retour arrière de transaction, continuation partielle ou arrêt contrôlé. La reprise n'est sûre que si elle est idempotente, dédupliquée par identifiant de commande, ou compensée. Il est incorrect de traiter automatiquement toutes les erreurs comme transitoires.

11.3 Transactions et effets externes

Une transaction locale peut offrir atomicité si le backend la garantit. Pour une suite de systèmes externes qui ne partagent pas une transaction distribuée, le modèle doit utiliser une orchestration de compensation (saga) ou déclarer explicitement les effets irréversibles. Un simple rollback de mémoire ne peut pas annuler un courriel envoyé, une commande physique ou une écriture distante déjà confirmée.

11.4 Reprise après interruption

Un checkpoint sauvegarde état sérialisé, version de schéma, registre de composants, derniers événements confirmés, budgets, empreintes d'artefacts et identifiants d'opérations externes. Au redémarrage, l'exécuteur valide le checkpoint, reconstruit les dépendances et détermine quels effets ont été confirmés. Les actions non décidables ne sont pas rejouées aveuglément : elles entrent dans un état de réconciliation. La continuité de travail exige une opération atomique de sauvegarde, des copies versionnées et un journal append-only.

11.5 Disponibilité dégradée et résultat partiel

Quand un composant devient indisponible, l'orchestrateur recherche une voie alternative compatible avec le contrat et les capacités disponibles, réutilise des résultats certifiés encore valides ou renvoie un résultat partiel explicitement étiqueté. La résilience préserve autant que possible la fonction; elle ne maquille pas la perte de garanties. Un fallback n'est accepté que si son contrat couvre la demande ou si la sortie expose précisément l'écart.

Invariant 11.1 (Échec explicite et non-promotion)**.** Aucun résultat marqué partiel, inconnu, périmé, non vérifié ou interrompu ne peut être converti en résultat certifié sans une nouvelle étape de vérification dont les préconditions sont satisfaites.

12 Composition de données, mémoire et provenance

Chaque valeur produite est rattachée à : composant, version, paramètres normalisés, hash des entrées pertinentes, contexte, temps, effets, artefacts de sortie, statut et dépendances. Un graphe de provenance $G = (V, E)$ lie une sortie à ses prémisses et à ses étapes de transformation. Les arêtes distinguent au moins dépendance de valeur, dépendance de contrôle, dépendance de preuve et dépendance d'environnement.

Proposition 12.1 (Propagation conservatrice d'invalidation)**.** *Soit un graphe de dépendances orienté G où $u \to v$ signifie que la validité de v dépend de u. Si un nœud u est révoqué, l'ensemble des descendants accessibles de u est marqué à revalider ou invalider avant qu'un de ces nœuds soit réutilisé comme prémisse certifiée.*

Proof. Par définition de l'arête, la garantie de validité de chaque descendant dépend au moins indirectement de u. Après révocation de u, la conjonction d'hypothèses qui justifiait cette garantie n'est plus établie. En parcourant le graphe dans l'ordre topologique, chaque descendant est donc placé en attente de revalidation; une promotion ultérieure nécessite une preuve alternative ou la reconstruction de ses dépendances valides. □

Cette règle suppose que les dépendances sont exhaustives et correctement enregistrées. En présence de cycles, les composants fortement connexes sont traités comme des unités de révision; toute dépendance cachée non enregistrée constitue une violation de l'invariant de provenance.

13 Mémoire vivante composable : état, compression, veille et réveil

13.1 Mémoire comme composition de services et de représentations

La mémoire n'est pas une boîte monolithique ni une simple liste de faits. On la définit comme une composition $\mathcal{M} = \langle I, S, P, D, A, R, V \rangle$, où I est l'index de recherche, S le stockage durable, P le graphe de provenance, D le graphe de dépendances, A le gestionnaire d'activation, R le mécanisme de révision/révocation et V les vérificateurs de fraîcheur et d'intégrité. Ces sous-fonctions sont elles-mêmes composables et remplaçables sous contrats. La mémoire conserve aussi bien des données

que des compositions, programmes, preuves, hypothèses, contre-exemples, épisodes, expériences, politiques, états de reprise et versions.

Un objet mémorisé est décrit par

$$m = (id, kind, payload, encoding, loss, prov, deps, scope, version, utility, cost, life),$$

où *loss* indique le caractère exact, sans perte, ou approximatif de l'encodage. Un résumé avec perte ne remplace jamais silencieusement la source : il pointe vers elle, décrit les distinctions préservées, l'erreur connue ou inconnue et les tâches pour lesquelles il peut être utilisé. Les certificats, événements de révocation, hashes et données nécessaires à l'audit ont une politique de conservation plus stricte que les caches reconstructibles.

13.2 États du cycle de vie

Un objet ou une composition peut être ACTIVE, IDLE, SLEEPING, ARCHIVED, STALE, REVALIDATING ou REVOKED. SLEEPING signifie que sa représentation active a été déchargée mais que sa description canonique, sa provenance et le contenu récupérable restent persistés. ARCHIVED peut déplacer une donnée vers un stockage plus froid et un encodage plus compact; cette transition est sans perte pour les objets qui portent une obligation de fidélité exacte. STALE interdit l'emploi comme prémisse active tant que les dépendances et le contexte n'ont pas été revalidés.

La machine d'états autorise notamment

$$\begin{aligned} \text{ACTIVE} &\xrightarrow{\text{inactivité + checkpoint}} \text{SLEEPING} \\ \text{SLEEPING} &\xrightarrow{\text{demande}} \text{REVALIDATING} \\ \text{REVALIDATING} &\xrightarrow{\text{hash/version/contrats valides}} \text{ACTIVE}. \end{aligned}$$

Un réveil ne signifie pas qu'un objet périmé redevient valide par simple chargement. Si une version d'une dépendance, un contexte, un droit ou une source a changé, le système doit revalider, reconstruire les descendants ou maintenir le statut stale. Les objets épinglés, les objets sous bail d'exécution, les preuves référencées par une transaction active et les événements de révocation ne sont pas évincés par une politique générique de mémoire.

13.3 Politique d'activation et utilité contextuelle

Pour une composition c, le contrôleur estime une valeur contextuelle $U(c \mid x)$, un coût de réveil $C_w(c)$, un coût de maintien actif $C_a(c)$, une probabilité de réutilisation $p(c \mid x)$ et un risque de péremption $R_s(c)$. Une politique d'éviction peut ordonner les candidats par une fonction déclarée, par exemple

$$J_{sleep}(c \mid x) = \lambda_a C_a(c) - \lambda_u p(c \mid x) U(c \mid x) + \lambda_r R_s(c),$$

avec coefficients/version inscrits dans la provenance. Il s'agit d'une politique de gestion de ressource, pas d'une vérité universelle : ses paramètres sont évalués selon le workload. Les objets sous obligation de conservation exacte sont hors du domaine d'éviction destructive. La mémoire garde les évictions, réveils, erreurs, latences et tailles réelles afin de réviser la politique sur des mesures.

Proposition 13.1 (Conservation au cycle veille-réveil)**.** *Supposons qu'un payload soit sérialisé selon une représentation canonique versionnée, persisté sans perte, et associé à un hash vérifié au retour. Supposons aussi que les versions de ses dépendances sont inchangées, ou que le vérificateur indépendant accepte explicitement leur révision. Alors, sous l'hypothèse opérationnelle de résistance aux collisions de la fonction de hash, le réveil restitue le même payload canonique ou refuse explicitement la restauration; il ne produit pas silencieusement un payload différent sous la même identité.*

Proof. Le chargement restaure les octets persistés, vérifie le hash et compare les versions. Si les checks passent, la représentation canonique restaurée possède la même empreinte que celle enregistrée; sous l'hypothèse de résistance aux collisions, cela soutient l'identité des octets. Si un check échoue, la transition est refusée. Les hypothèses de version et de revalidation empêchent d'assimiler l'intégrité des octets à la fraîcheur sémantique. □

Cette proposition ne garantit pas à elle seule la vérité du contenu, la sûreté du stockage ni l'impossibilité de collusion entre producteurs et vérificateurs.

13.4 Compression structurale, compression de contenu et réversibilité

Trois opérations doivent être distinguées : (i) *factorisation structurale*, qui remplace des sous-graphes identiques par une référence partagée sans changer leur sémantique; (ii) *compression sans perte*, réversible au niveau des octets ou d'une représentation canonique; (iii) *résumé avec perte*, qui conserve certaines propriétés selon un contrat et ne peut pas être utilisé comme source exacte. La factorisation ne peut fusionner des nœuds que si l'identité de contexte, d'effets, de version et de trace l'autorise. La compression contextuelle doit annoncer le domaine des questions auxquelles elle répond correctement; une demande qui dépasse ce domaine réveille la source exacte.

La mémoire peut ainsi se « contracter » pour un contexte, mettre en veille les sous-compositions sans demande actuelle, partager les structures répétées, puis se « dilater » par réveil à la demande. La perte de détails n'est permise que dans une vue dérivée explicitement marquée; les données et preuves sources restent récupérables selon leur politique de conservation.

13.5 Propagation sous forme de vagues d'informations

Une vague est un événement typé $w = (id, src, ver_s, dst, kind, payload, ctx, priority, ttl, seq, prov)$. Le contrôleur propage les deltas vers les abonnés concernés plutôt que de dupliquer systématiquement toute la mémoire. La transmission utilise la déduplication par identifiant, un TTL ou nombre de sauts, une limite de fan-out, une priorité, un ordre causal si nécessaire et une file persistante à backpressure. Les événements critiques – révocation, invalidation de certificat, changement de version, violation de sécurité – sont durables et ne peuvent pas être jetés par expiration de cache. Les événements d'exploration à faible valeur peuvent expirer avec preuve de l'expiration et lien vers la source.

Le mot « vague » désigne ici une diffusion événementielle bornée et traçable, pas une propagation physique ni un mécanisme qui dispense de contrôler les dépendances. Les cycles de diffusion sont arrêtés par la déduplication et le budget de sauts; les destinataires valident la version, le contexte et la provenance avant d'actualiser leur état.

13.6 Invariants de mémoire vivante

1. **Intégrité :** payload, version et hash concordent avant tout réveil.
2. **Provenance :** toute donnée dérivée conserve la chaîne de source et de transformation.
3. **Dépendances :** une modification invalide ou revalide les descendants transitivement; les cycles du graphe de dépendances sont interdits ou traités par un protocole explicitement défini.
4. **Conservation :** une éviction ne supprime pas un objet requis par un contrat, une preuve active, une transaction ou un audit.
5. **Freshness :** toute donnée périmée est étiquetée et ne sert pas de prémisse certifiée.
6. **Bounded propagation :** vagues et réveils respectent le budget, la déduplication, le backpressure et la priorité des invalidations.
7. **Réversibilité :** un résumé approximatif ne détruit pas la référence à sa source; une transformation destructrice exige une décision explicite et une trace.

14 Synthèse, recherche et transformation de programmes

14.1 La composition comme objet de recherche

Le système ne doit pas seulement exécuter des compositions rédigées à l'avance. Il doit pouvoir rechercher une composition p dans une grammaire $\mathcal{G}$ telle que, pour un jeu d'exemples d'entraînement D_{tr}, p satisfasse les contrats et minimise un objectif vectoriel :

$$J(p) = (L_{ex}(p), L_{contraintes}(p), L_{transfert}(p), C(p), R_{effets}(p), U(p)),$$

où les termes représentent respectivement l'écart aux exemples, les violations de contraintes, la perte de transfert, le coût, le risque lié aux effets et l'incertitude estimée. Les termes incompatibles restent séparés jusqu'à ce que la politique de décision justifie un arbitrage. Une pénalité basse ou une solution courte n'établit pas sa correction.

14.2 Boucle de synthèse

La boucle de synthèse comporte : formalisation du besoin; définition des observables et de l'oracle de test; génération de programmes dans une grammaire typée; normalisation des formes équivalentes; vérification statique; exécution sur cas entraînement; recherche de contre-exemples; raffinement; tests holdout; tests perturbés; analyse d'effets; certification conditionnelle; archivage du programme et de sa provenance. Le jeu holdout reste séparé du générateur et du normaliseur; les réponses attendues ne peuvent pas devenir des variables d'entrée de synthèse.

14.3 Décomposition et réutilisation hiérarchique

Un programme complexe peut être factorisé en sous-compositions nommées avec contrat, version et coût, puis recomposé à un niveau supérieur. Une décomposition utile doit conserver le comportement observable annoncé. La factorisation n'est pas correcte simplement parce qu'elle produit du code plus court; elle doit préserver les postconditions, les effets, l'ordre des événements et les garanties de ressources exigées.

14.4 Composition évolutive et auto-transformation

Un programme peut proposer des transformations de lui-même ou de ses sous-compositions : remplacement d'un composant, factorisation, fusion, parallélisation, spécialisation, généralisation, changement de stratégie de recherche, ajout d'une étape de validation, ou réorganisation des budgets. Ces transformations sont des candidats. Pour être mises en service elles doivent satisfaire un contrat de transformation, produire un diff, passer les vérifications et tests, être comparées sur un ensemble de régression, et être livrées avec la possibilité de rollback. La version actuellement active reste identifiable, et une dégradation détectée entraîne la restauration du dernier état approuvé.

14.5 Généralisation compositionnelle

Le test de compositionnalité doit séparer les briques vues et les compositions vues. Un split pertinent tient les primitives disponibles mais réserve des combinaisons inédites; des splits plus stricts réservent aussi des familles de primitives et des contextes. Les métriques comprennent l'exactitude sur composition inédite, la profondeur atteinte, la sensibilité à l'ordre, la robustesse aux noms permutés, la réutilisation correcte de contrats, le taux de violation de contraintes et le coût de recherche. Un test qui contient des gabarits presque identiques au jeu d'entraînement mesure surtout la mémorisation.

15 Explosion combinatoire : recherche, activation et réutilisation

15.1 Diagnostic de complexité

Si une grammaire fournit en moyenne b prolongements possibles à chaque nœud et que la profondeur de recherche est d, une énumération naïve peut produire de l'ordre de $\sum_{i=0}^{d} b^i$ arbres avant élimination; dans le pire cas, la taille de l'espace candidat croît exponentiellement avec la profondeur. Un mécanisme de veille peut réduire l'empreinte active et les coûts de résidence, mais ne réduit pas à lui seul ce nombre d'arbres, le nombre d'expansions ni le coût de vérification. Il répond à un sous-problème – mémoire active – et doit être couplé à des mécanismes de réduction de recherche.

15.2 Défense multicouche contre l'explosion

Le moteur de synthèse applique les mécanismes compatibles avec le contrat de tâche :

1. **Génération paresseuse :** ne matérialiser les branches qu'au moment où le front de recherche les développe.
2. **Filtrage précoce :** éliminer avant exécution les candidats mal typés, incompatibles avec les contrats, violant les effets autorisés, dimensions d'unités ou préconditions.
3. **Recherche best-first / anytime :** traiter d'abord les candidats selon un score déclaré mêlant coût, contraintes satisfaites, information attendue et preuve disponible; ce score guide l'ordre, jamais la certification.
4. **Partage de sous-structures :** conserver les sous-compositions réutilisables comme DAG et partager un sous-calcul uniquement quand la sémantique, le contexte, les effets et les entrées observées coïncident.
5. **Normalisation et quotient d'équivalence :** ne fusionner deux candidats que si une équivalence de syntaxe normalisée ou une équivalence comportementale prouvée dans un domaine déclaré le permet. Une similarité numérique n'est pas une preuve d'équivalence.
6. **Boucle contre-exemple-guidée :** tester une composition candidate, chercher les plus petits contre-exemples, ajouter les contraintes produites au problème de synthèse et reprendre le front sans effacer l'historique.
7. **Veille/réveil :** sortir de la mémoire active les compositions inactives tout en conservant le descripteur, le payload durable, le hash, la provenance, les dépendances et le checkpoint du travail.
8. **Budgets et continuation :** toute frontière de budget préserve le front, les signatures déjà vues, les graines et les statistiques. BUDGET_EXHAUSTED_RESUMABLE n'est ni un échec sémantique ni la preuve d'absence de solution.

15.3 Ordonnancement juste et reprise

Chaque entrée du front est $q = (ast, signature, cost, depth, priority, dependencies, proofState)$. L'ordonnanceur conserve un ordre stable à priorité égale et n'autorise pas la famine permanente d'une classe de tâches : une politique peut réserver une fraction du budget aux candidats anciens, à la vérification et aux tâches de maintenance. Les checkpoints du front incluent la grammaire et ses versions, les coûts, les signatures, le registre des candidats éliminés avec raisons, l'état aléatoire, les compteurs et les dépendances de preuves. Les paramètres de recherche font partie de l'expérience reproductible.

Proposition 15.1 (Équité de recherche sous hypothèses)**.** *Supposons que chaque candidat de rang fini ait un nombre fini de prédécesseurs dans l'ordre retenu, que les extensions par profondeur soient finies, que les budgets puissent être repris sans perte, que l'ordonnanceur soit équitable et que des*

ressources positives continuent d'être affectées à la recherche. Alors tout candidat atteignable à rang fini est éventuellement retiré du front et examiné.

Proof. Pour un candidat q, l'ensemble de ses prédécesseurs est fini par hypothèse. La politique équitable finit par examiner chacun d'eux, donc par générer le prochain préfixe de la chaîne menant à q. La reprise sans perte conserve ces préfixes entre deux budgets. Une induction sur la longueur minimale du chemin de génération établit que q finit par entrer dans le front puis en est retiré. Cette propriété est une propriété de complétude à rang fini, pas une borne de temps ni une garantie de trouver une solution dans tout langage infini. □

15.4 Rôle précis du sommeil et du réveil

Le sommeil traite la pression mémoire et les coûts de maintien : les objets actifs restent dans le working set; les compositions inactives sont sérialisées, compressées sans perte, indexées et déchargées; le réveil est déclenché par requête, dépendance, événement ou stratégie de planification. Il ne faut pas endormir un bloc sous bail d'exécution, une donnée épinglée, une invalidation prioritaire non propagée, un checkpoint non confirmé ou un objet requis pour l'audit. Au réveil, l'intégrité est vérifiée et la fraîcheur est recontrôlée. Le coût réel des cycles veille/réveil est mesuré; les objets qui se réveillent trop souvent peuvent rester résidents temporairement. La politique doit pouvoir être adaptée aux ressources réellement disponibles.

15.5 Mesures spécifiques

Outre l'exactitude, mesurer le nombre de candidats générés/typiquement rejetés par couche, le taux de partage de sous-arbres, la taille du front, les doublons évités, l'efficacité du quotient d'équivalence, la mémoire résidente, l'espace froid, la latence réveil/hibernation, la durée moyenne avant réveil, le coût CPU total, le ratio solutions validées/candidats et la qualité des solutions sous budget égal. Comparer la veille désactivée/activée sur les mêmes tâches; sinon un gain mémoire peut masquer un surcoût de recherche ou de latence.

16 ASCRC : régulation réversible de l'activation compositionnelle

16.1 Séparation inviolable avec ASCR d'eGenBrain v4

ASCR d'eGenBrain v4 conserve exactement son rôle, ses interfaces et sa sémantique propres dans le périmètre décrit par ses sources. La présente théorie ne redéfinit pas ASCR, ne remplace aucune de ses fonctions, ne change pas ses données internes et ne lui ajoute aucun effet implicite. Le mécanisme nouveau est nommé **ASCRC** et constitue un régulateur architectural distinct. ASCRC peut recevoir des descripteurs de candidats provenant d'ASCR ou d'autres catalogues via une interface d'adaptation non destructive; il ne modifie ni ne supplante le composant source.

16.2 Les dimensions qu'il ne faut jamais confondre

Pour une composition c, on distingue au moins les variables suivantes :

$$E(c) \in \{0,1\}, \quad K(c) \in \mathcal{K}, \quad L(c) \in \mathcal{L}, \quad A(c) \in \{0,1\}, \quad X(c) \in \{0,1\}.$$

E indique l'existence dans le catalogue; K le statut épistémique (hypothèse, testée, certifiée, réfutée, etc.); L le cycle de vie de résidence (actif, dormant, archivé, périmé, révoqué); A l'admission opérationnelle à une politique/context; et X l'activation computationnelle en cours. L'exécution est un événement distinct qui exige $X = 1$, des droits, un contrat admissible et des ressources. Aucune transition d'activation, suspension, allocation ou réveil ne modifie $K(c)$ par elle-même.

Définition 16.1 (Descripteur ASCRC)**.** Un descripteur est un tuple versionné

$$d_c = (id, ver, \sigma, \Gamma, Deps, Scope, Cost, Risk, U, N, I, Age, Pin, Lease, WakeOn, K, Prov),$$

où σ est la signature canonique, Γ le contrat, *Deps* les dépendances, *Scope* le domaine et contexte, *Cost* le coût estimé, *Risk* les risques, U l'utilité contextuelle estimée, N la nouveauté, I l'information discriminante attendue, *Age* le temps depuis la dernière activation, *Pin/Lease* les protections contre l'éviction, $WakeOn$ les déclencheurs de réveil, K le statut épistémique préexistant et *Prov* la provenance. Les estimations sont des métadonnées décisionnelles, non des certificats.

16.3 Contrat fonctionnel et états de sortie

ASCRC prend en entrée un objectif versionné G, le contexte x, un flux ou catalogue de descripteurs, un vecteur de ressources disponibles B, l'état persistant de recherche et les événements récents. Il rend une décision traçable : ensemble admissible, priorités, allocations, compositions à activer, compositions à mettre en veille, réveils déclenchés, raisons de report, état de continuation et compteurs de ressource. Les états opérationnels distinguent au minimum CATALOGUED, ELIGIBLE, ACTIVE, DORMANT, STALE, BLOCKED et REVOKED. BLOCKED et DORMANT ne signifient pas réfuté; REVOKED requiert un événement de révocation distinct, motivé et traçable.

16.4 Budget et politique d'activation : complémentarité, pas substitution

Un budget est un plafond dur de ressource physique; ASCRC est une politique d'allocation de ce plafond. Pour un vecteur $B = (B_{cpu}, B_{mem}, B_{time}, B_{io}, B_{external})$ et des allocations b_i :

$$\sum_{i \in A_t} b_i^{(r)} \leq B_t^{(r)} \qquad \text{pour chaque ressource } r.$$

ASCRC ne supprime donc pas le besoin de budgets, de limites de concurrence ou de contrôle d'effets. Une commande peut être « activée par ASCRC » sans spécifier un budget local ad hoc, mais le runtime conserve toujours des limites globales et des garde-fous de capacité. Le budget définit ce qui peut être dépensé; ASCRC choisit sur quoi le dépenser et quand différer le travail.

16.5 Allocation multi-critère et exploration protégée

Une priorité indicative peut prendre la forme

$$S(c \mid G, x) = w_u U + w_n N + w_i I + w_a \phi(Age) + w_d Diversity - w_c C - w_r R,$$

avec paramètres versionnés et normalisation déclarée. Les poids ne sont pas universels; ils sont calibrés par protocole et comparés à des politiques de référence. Le score ne signifie ni vérité ni validité. Les contraintes dures (type, contrat, permissions, dépendances, sécurité et statut stale) ne peuvent être compensées par un score élevé.

L'allocation réserve selon la tâche des parts distinctes pour exploitation, exploration, vérification et maintenance :

$$B_t = B_t^{exploit} + B_t^{explore} + B_t^{verify} + B_t^{maint},$$

avec somme composante par composante au plus égale à B_t. Tant que des alternatives pertinentes restent non résolues, le contrôleur peut imposer un quota d'exploration positif ou une politique de couverture stratifiée. Le quota peut être désactivé seulement si une raison formelle liée à l'objectif le justifie et est inscrite dans la trace; il ne peut pas supprimer une porte obligatoire de sûreté.

16.6 Filtrage hiérarchique pour ne pas déplacer l'explosion vers ASCRC

Le calcul de priorité de tous les candidats peut lui-même devenir coûteux. ASCRC ne doit donc pas matérialiser ni scorer intégralement un espace potentiellement immense. La politique cible est hiérarchique : (0) génération paresseuse et vérification syntaxique/type/contrat; (1) indexation par signature, dépendance, domaine et cache; (2) score léger sur les seuls candidats qui entrent dans le front admissible; (3) test, probe ou simulation plus coûteuse pour un sous-ensemble discriminant; (4) activation/exécution réelle; (5) mise à jour de la politique à partir des résultats et contre-exemples.

Chaque étage donne un résultat typé et son coût est mesuré. Un candidat non matérialisé reste représenté par l'état du générateur et son checkpoint, afin de rendre la recherche reprenable.

16.7 Veille, réveil et non-destruction

La transition ACTIVE → DORMANT décharge la représentation résidente après checkpoint et persistance du descripteur, du payload récupérable, des versions, des dépendances, de la trace et de l'état de recherche. Elle ne supprime ni la composition ni son statut épistémique. La transition DORMANT → ACTIVE exige demande ou événement, intégrité du payload, compatibilité du schéma, contrôle des versions et des permissions, revalidation des dépendances et réservation effective des ressources. Une dépendance modifiée produit STALE jusqu'à résolution explicite.

Les déclencheurs incluent notamment nouvelle observation, changement d'objectif ou de contexte, contre-exemple, échec d'une composition active, disponibilité d'une dépendance, requête explicite, nouvelle preuve, expiration d'un bail et alarme de maintenance. Une composition épinglée, louée, en transaction ouverte, indispensable à une révocation ou portant un état non sauvegardé n'est pas endormie par éviction ordinaire.

16.8 Anti-famine, diversité et limites de la garantie

ASCRC combine vieillissement borné, priorité aux tâches anciennes, quota d'exploration, rotation par familles et budgets de vérification. Pour un ensemble fini de candidats admissibles, si chaque candidat a un coût inférieur ou égal à la part de ressource disponible à terme, si l'ordonnanceur est appelé indéfiniment, si le quota de maintenance/exploration reste positif et si les checkpoints reprennent sans perte, un candidat admissible ne peut être reporté indéfiniment. Cette garantie ne s'étend pas sans hypothèse à un espace infini alimenté plus vite que le service, à un candidat dont le coût dépasse toute capacité disponible ou à des objectifs qui changent à chaque pas; ces situations requièrent une politique de classes, de fractionnement, de coût maximal ou de réponse explicite de non-admissibilité actuelle.

Proposition 16.2 (Absence de famine sur un ensemble fini)**.** *Soit S un ensemble fini de candidats continuellement admissibles. Supposons que chaque candidat ait un coût inférieur ou égal à la capacité allouable lors d'une ronde de service, qu'aucune protection (pin, lease, transaction) ne l'exclue, que le contrôleur soit exécuté indéfiniment, qu'une part positive des rondes serve la classe concernée, et que l'ordonnancement sélectionne en priorité les candidats de cette classe par âge décroissant avec remise à zéro de l'âge servi. Alors aucun candidat de S ne peut être différé indéfiniment; à partir du moment où la classe est continuellement éligible, chaque candidat finit par être activé.*

Proof. À chaque ronde de service, un candidat sélectionné voit son âge remis à zéro; chaque candidat continuellement admissible et non sélectionné vieillit. L'ensemble S étant fini, il existe à chaque ronde un candidat d'âge maximal. La règle de priorité empêche un candidat récemment servi de passer indéfiniment devant un candidat plus ancien. Si un candidat restait indéfiniment sans service, son âge finirait par dépasser celui de tout candidat sélectionné dans la classe; l'ordonnanceur devrait alors le sélectionner, contradiction. La conclusion dépend des hypothèses de coût, d'admissibilité continue et de rondes de service; elle ne couvre pas un espace infini ni un flot d'arrivées non borné. □

Proposition 16.3 (Invariance épistémique sous régulation ASCRC)**.** *Si une transition ASCRC se limite à l'admission, à l'allocation, à l'activation, au report, à la veille ou au réveil et conserve les objets épistémiques et leurs dépendances, alors elle préserve le statut épistémique $K(c)$ de chaque composition. En particulier, ni une priorité nulle ni la mise en veille ne peuvent produire* REFUTED *ou* CERTIFIED*.*

Proof. Par définition, ces transitions modifient seulement les champs opérationnels A, L, X et les allocations. Le champ K est en lecture seule dans le contrat ASCRC; toute modification de K exige

un événement séparé émis par un processus d'observation, d'évaluation, de réfutation, de preuve ou de révocation et portant sa propre provenance. Donc une transition ASCRC seule ne peut changer $K(c)$. □

Invariant 16.4 (Non-déstruction par régulation)**.** Pour tout candidat découvert et durablement enregistré, le report ou la veille ASCRC conserve au minimum son identifiant, sa version, sa signature, son statut épistémique, sa provenance, ses dépendances et son pointeur vers son payload ou son checkpoint, sauf politique de conservation destructive explicitement autorisée, distincte d'ASCRC, auditée et compatible avec les obligations légales/de sûreté applicables.

16.9 Métriques et protocole expérimental ASCRC

Évaluer à tâche, graines et budget physique appariés : nombre de candidats générés, matérialisés, scorés, activés et exécutés; taux de candidats dormants réveillés utiles; taux de réveils superflus; coût ASCRC par activation; taille du front; débit de candidats validés; diversité des familles visitées; âge maximum en attente; taux de famine; consommation CPU/mémoire/I/O; qualité finale et erreurs de sûreté. Comparer au moins : ordonnanceur FIFO, best-first seul, ASCRC sans quota exploration, ASCRC avec quota, politique aléatoire stratifiée et ASCRC sans veille. Mesurer les interactions : ASCRC peut réduire les activations simultanées, mais son surcoût de scoring et le coût de vérification doivent être comptés. Aucun nombre de réduction n'est affirmé avant l'expérience.

16.10 Relation formelle avec la programmation compositionnelle

ASCRC n'est pas un privilège extérieur au langage : ses étapes de filtrage, classement, allocation, veille, réveil et vérification sont des compositions de composants typés, soumises aux mêmes contrats, versions, effets et traces. Le contrôleur ASCRC est méta-niveau : il sélectionne un petit ensemble d'opérations à activer sans changer la sémantique des programmes sélectionnés. Sa propre politique est versionnée, testée contre une politique de référence et peut être révisée avec retour arrière.

17 Programmation compositionnelle multimodale et sémantique

Le noyau traite uniformément les contrats, mais garde les types de domaine distincts. Une composition de langage peut lier tokenisation déterministe, parseur, analyse de référents, interprétation de portée, mémoire contextuelle, résolution d'ambiguïté, contrôle d'entailment et vérificateur. Une composition numérique peut lier parseur d'expression, système d'unités, simplificateur, solveur et vérificateur de preuve. Une composition d'observation peut lier collecte autorisée, empreintes multi-vues, modèle de transition, sondes et comparaison de prédictions. Tous sont des plans de composition différents sur un même cadre de contrats; aucune couche linguistique ne bénéficie d'un passe-droit épistémique.

Les idées multirésolutions sont transférables sous forme de vues ou projections hiérarchiques sur les données (octets, tokens, syntagmes, structure, sémantique, trajectoire) uniquement si chaque vue apporte une utilité testable et conserve sa provenance. Aucune architecture de FulleGenBrain ne dépend d'un modèle génératif de langage ou de poids d'un tel modèle. Les capacités de formulation et de découverte proviennent de représentations explicites, de données observées, de grammaires de programmes, de recherche, d'analogies calculables, de moteurs spécialisés et de composition.

18 Programmation par composition comme pilier de FulleGenBrain_4

La programmation compositionnelle n'est pas un service externe dans FulleGenBrain_4 : elle est le mécanisme structurel par lequel le système assemble ses voies cognitives, transforme ses plans de calcul, apprend de nouveaux comportements et répare ses propres chaînes d'action. Les composants

fondamentaux sont des compétences (observer, parser, calculer, chercher, mesurer, induire, simuler, vérifier), des méthodes (stratégies réutilisables), des règles/invariants (contraintes transversales) et des adaptateurs de domaine. Les compositions sont des objets typés enregistrés en mémoire et exécutables par un runtime gouverné.

18.1 Architecture à trois plans

1. **Plan de définition :** catalogue versionné, contrats, DSL/AST, types, effets, budget, politiques d'erreur, schémas de provenance et tests.
2. **Plan d'exécution :** interpréteur de référence, compilateur, ordonnanceur, contrôleur d'état, sandbox par capacités, journal et checkpoints.
3. **Plan d'évolution :** induction de candidats, découverte de sous-programmes, synthèse, test différentiel, évaluation sur holdout, approbation ou retour arrière.

Les plans sont séparés mais reliés par un identifiant de composition et un contrat commun. L'évolution peut proposer les changements; seul le chemin de validation déterminé peut activer une version nouvelle.

18.2 Répartition des responsabilités

Le contrôleur décide quel objectif poursuivre et quand s'arrêter; il n'exécute pas implicitement chaque compétence. Le synthétiseur propose des arbres de composition; il ne les certifie pas. Le compilateur contrôle que le programme est bien formé et produit un artefact reproductible; il ne certifie pas le résultat métier. Le runtime exécute uniquement les composants résolus par registre et les effets autorisés. Le vérificateur évalue les critères indépendamment de l'heuristique de recherche. La mémoire conserve artefacts, essais, dépendances et versions; elle ne convertit pas une sortie en vérité. Le gestionnaire de résilience observe l'état de santé des composants et choisit les reprises déclarées.

18.3 Interfaces minimales des blocs

Tout bloc doit déclarer : identifiant/version; entrée/sortie typées; préconditions/postconditions; effets et capacités; coût prévu et limite; exceptions/résultats partiels possibles; stratégie de compensation ou raison de non-compensabilité; provenance; tests unitaires; vérificateur; règles de compatibilité; comportement de révocation. Une ancienne version demeure disponible tant qu'elle est référencée par une composition active, un checkpoint ou une preuve historique.

19 Modèle du monde : rôle fonctionnel, preuve d'utilité et activation conditionnelle

19.1 Rôle précis et domaine d'applicabilité

On entend ici par modèle du monde un modèle de transitions qui prévoit un état, un événement ou une récompense conditionnellement à une action : $\hat{s}_{t+1}, \hat{r}_t = \hat{T}(s_t, a_t, c_t)$. Sa fonction candidate est d'évaluer des actions et branches sans les exécuter toutes dans le monde réel, de produire des contre-factuels, de prévoir des conséquences multi-pas, ou de guider une sonde/action. Il n'est pas nécessaire par principe pour une dérivation mathématique pure, une simple transformation déterministe entièrement spécifiée, ou une tâche où il ne change ni les choix ni la qualité du résultat. Il peut apporter une utilité dans des problèmes dynamiques, coûteux, interactifs et observables; cette utilité est une hypothèse à mesurer.

19.2 Constats issus des sources Order 62/63

Le dossier de revue FulleGenBrain_1 contient un extrait de `rapport\_experience\_model\_order\_62.tex` et de `rapport\_experience\_model\_order\_63.tex`. Le rapport O62 indique, pour sa campagne décrite, base : 3 `PASS`, 4 `FAIL`, 11 `EXHAUSTED`; variante RS : 2 `PASS`, 4 `FAIL`, 12 `EXHAUSTED`; récompense moyenne rapportée : 0,6833 contre 0,1972. Il conclut que RS ne doit pas être promu et reste optionnel/désactivé par défaut dans ce snapshot. O63 rapporte un MSE latent holdout de $1{,}7310954885 \times 10^{-4}$ sur 320 problèmes, 1436 transitions et 27 actions opaques, mais note aussi des résultats hétérogènes; dans la comparaison RS dédiée, symbolic reste PASS dans les deux bras, tandis que routing passe de PASS au baseline à EXHAUSTED avec RS. O63 conclut que le navigateur apprend une structure mais que son bénéfice décisionnel général n'est pas établi comme bénéfique.

Ces observations ne prouvent ni que tout modèle du monde est inutile, ni qu'il est utile en général. Elles illustrent précisément la séparation entre qualité prédictive d'un substitut et utilité causale dans la décision. Certaines ablations d'O62 sont rapportées comme non discriminantes, ce qui interdit une conclusion causale forte pour les composants concernés. Ces nombres sont des résultats rapportés dans le corpus historique; ils n'ont pas été réexécutés ici dans leur environnement complet. Les détails de provenance sont enregistrés dans le dossier de preuves FulleGenBrain_4.

19.3 Porte d'activation conditionnelle

Pour une famille de tâches D, comparer sur les mêmes instances, graines, budgets et oracles au moins les bras suivants :

1. B_0 : aucune prédiction de modèle du monde;
2. B_1 : modèle réel appris ou spécifié, avec trace d'utilisation;
3. B_2 : contrôle nul ou sorties permutées, conservant le coût d'appel;
4. B_3 : modèle ablaté ou prédiction d'un pas seulement, pour identifier quel aspect est utile;
5. B_4 : modèle idéal/oracle, uniquement si l'oracle est licite et isolé, comme borne de diagnostic et non comme comparateur équitable de déploiement.

L'évaluation porte sur réussite vérifiée, coût total, qualité de l'action choisie, nombre d'expériences réelles économisées, erreurs de prédiction, stabilité sous perturbation, généralisation OOD, calibration et taux de décisions détériorées. Les comparaisons sont appariées; les intervalles d'incertitude, l'effet minimal pratiquement utile et la règle d'activation sont fixés avant de regarder le jeu test.

Une politique contextuelle active le modèle seulement si un gain décisionnel répliqué est supérieur au seuil préenregistré, compte tenu du coût de prédiction et du risque; si l'effet est nul, incertain, négatif ou trop coûteux, le contrôleur utilise une autre voie sur cette tâche tout en conservant la capacité du modèle pour les contextes où elle pourra être utile. Une moyenne positive ne suffit pas si une famille importante est fortement dégradée; les portes de sécurité et de certification restent dures.

$$\mathrm{UseWM}(D) = \mathbf{1}\{\mathrm{LCB}(\Delta U_D) > \delta_D \ \wedge \ \text{portes de sûreté satisfaites}\},$$

où $\Delta U_D = U(B_1; D) - U(B_0; D)$, LCB est une borne inférieure d'incertitude préspécifiée et δ_D le seuil de gain utile. Ce choix n'est pas une preuve du modèle; il s'agit d'une politique d'activation empirique versionnée.

19.4 Composition interne du modèle du monde

Le modèle du monde est lui-même une composition remplaçable : représentation d'état, modèle d'observation, transition/action, incertitude, simulateur de branche, comparateur et vérificateur post-action. Chaque sous-composant a une provenance et peut être ablaté. Les sorties simulées ont le statut PREDICTED ou SIMULATED, jamais OBSERVED ni CERTIFIED; une transition réelle

devient une observation et peut corriger ou révoquer les prédictions. Les décisions critiques gardent une voie indépendante de validation de l'état réel.

Invariant 19.1 (Non-promotion d'une prédiction)**.** Aucune prédiction du modèle du monde ne devient fait observé ou certificat du seul fait qu'elle est cohérente avec d'autres prédictions du même modèle.

20 Protocole de validation exhaustif et évolutif

La validation doit être une matrice de contrats, de composition et de tâches réelles, pas une série unique de démonstrations.

Niveau	Test exigé	Preuve conservée
Composant	typage, pré/postconditions, valeurs frontières, erreurs, effets	logs, tests, versions, coverage et hashes
Connexion	compatibilité de types, unités, contextes, provenance	rapport de connexions refusées/acceptées
Composition	séquence, branche, map/fold, parallèle, boucle, reprise, transaction	trace et état final reproductible
Transformation	sémantique avant/après, diffs, régression, fallback	rapport de comparaison et décision d'activation
Résilience	panne injectée, interruption, timeout, restauration, doublons	journal de récupération et effets réconciliés
Synthèse	exemples, contre-exemples, ambiguïté, holdout, OOD	candidats, budget, oracle séparé, métriques
Sécurité	privilège minimal, identifiant non enregistré, chemin invalide, exécution indirecte	refus explicites et test red-team
Intégration	charge, contention, mémoire, dérivations révoquées, versions mixtes	profils, incidents, reprise et résultat final

Les portes de sortie sont indépendantes : syntaxe/compilation, typage, contrat, sécurité des effets, intégrité de trace, qualité métier, généralisation et coût. Le résultat global rapporte le vecteur de scores et les échecs par catégorie. Les cas non exécutés sont marqués `NOT_RUN`; un budget épuisé signifie que l'essai est incomplet, pas que la capacité est absente. Tout cas sous-déterminé conserve plusieurs modèles ou demande une observation discriminante.

21 Obligations théoriques restantes et stratégie de fermeture

Une théorie complète et harmonieuse n'est pas acquise par la seule présence des définitions. Les obligations suivantes doivent être refermées progressivement par une formalisation et des artefacts testables :

1. Définir une logique de contrats et une théorie des types suffisamment expressive pour les unités, les contextes et les effets.
2. Spécifier la sémantique complète de chaque nœud DSL et démontrer la correspondance entre interpréteur et compilateur.
3. Formaliser concurrence, annulation, équité de l'ordonnancement et partage des ressources.
4. Formaliser transactions locales, sagas, opérations idempotentes et réconciliation post-crash.
5. Définir les règles de compatibilité/versionnement des composants et des compositions persistées.

6. Construire des preuves machine pour les lemmes prioritaires et un noyau de vérification minimal.
7. Mesurer empiriquement la capacité de synthèse et de généralisation sur des splits compositionalement stricts.
8. Fermer la chaîne d'invalidation pour les dépendances causales, sémantiques et d'environnement qui ne sont pas directement des dépendances de données.

Ces obligations sont un programme de travail positif, pas une prédiction d'échec. Si un type, un opérateur ou un cas d'usage nécessaire n'est pas pris en charge, le processus prescrit est de le spécifier, le formaliser, l'implémenter, le vérifier et l'intégrer, en maintenant un statut épistémique précis durant le travail.

22 Synthèse des garanties et des statuts

Propriété	Statut dans ce document	Portée exacte
Correction séquentielle sous contrats	PROVED-UNDER-ASSUMPTIONS	preuve mathématique fournie, hypothèses explicites
Indépendance parallèle	PROVED-UNDER-ASSUMPTIONS	uniquement pour effets et lectures/écritures disjoints
Propagation d'invalidation	PROVED-UNDER-ASSUMPTIONS	graphe exhaustif, descendants réévalués
DSL AAM avec 4 nœuds de base	SOURCE-OBSERVED	artefact de démonstration analysé
Compilateur AAM interdit les fragments source libres dans le DSL étudié	SOURCE-OBSERVED	implémentation présente, revue non exhaustive de sécurité
Équivalence compilateur/interpréteur complète	EMPIRICAL-TO-TEST	à couvrir par tests différentiels et preuve de traduction
Synthèse généralisant à des compositions inédites	EMPIRICAL-TO-TEST	aucune généralisation globale déduite du formalisme
FulleGenBrain_4 intégré de bout en bout	À CONSTRUIRE/ÉVALUER	ce document définit l'architecture cible, pas une exécution intégrée

23 Conclusion

La programmation compositionnelle devient un pilier lorsqu'elle fournit le langage de structure des capacités, le système de contrats, la sémantique d'exécution, la voie de synthèse et le mécanisme de transformation contrôlée. Sa solidité dépend de lois explicites, de préconditions compatibles, d'effets analysés, de preuves traçables, de tests adversariaux et de la capacité à reprendre après panne sans masquer les écarts. La meilleure architecture n'est pas celle qui interdit d'évoluer : elle est celle où chaque évolution est représentée, construite, examinée, testée, traçée et, si nécessaire, annulée proprement. Dans FulleGenBrain_4, la veille/réveil, la recherche reprenable, la mémoire composable et l'activation conditionnelle de modèles de transition sont des règles de premier rang, intégrées au même cadre de types, contrats, provenance et validation.